

Delta Reinforcement Learning-Guided Memetic Algorithm for the Capacitated Vehicle Routing Problem

V. Lalith Kumar Reddy, M. Chandra Vadhan, P. Raj Ishwar

Department of Information Technology
Indian Institute of Information Technology
Allahabad, Prayagraj, India

iit2023048@iiita.ac.in, iit2023042@iiita.ac.in, iit2023122@iiita.ac.in

Abstract—The Capacitated Vehicle Routing Problem (CVRP) is a classical NP-hard combinatorial optimization problem with widespread applications in logistics and transportation. Traditional Genetic Algorithms (GAs) for CVRP suffer from premature convergence and static hyperparameter configurations that fail to adapt to the evolving search landscape. In this paper, we propose a Delta Reinforcement Learning-Guided Memetic Algorithm (Delta RL-MA) that replaces conventional parameter-level RL tuning with a population-level Q-learning agent that dynamically selects mutation operators for the entire population at each generation. Unlike prior work that uses RL to adjust crossover and mutation *rates*, our agent observes aggregate population features—including total cost, route length variance, and capacity violations—through a 729-state discretized space, and selects from five specialized CVRP operators (swap, 2-opt, relocate, block swap, and block relocate). The algorithm further introduces an anti-stagnation “Panic Window” mechanism that forcefully diversifies the population when plateaus are detected. We evaluate our approach on 12 standard CVRP benchmark instances from the TSPLIB library and compare against the RL-GA framework of Quevedo et al. (GECCO 2021). Experimental results demonstrate that our Delta RL-MA achieves or matches the best-known optimal solution on 11 of the 12 compared instances, improves upon the base RL-GA on 10 out of 12 instances in average cost, and achieves significantly lower standard deviations (average 62% reduction), indicating superior consistency and robustness across the evaluation set.

Index Terms—Capacitated Vehicle Routing Problem, Reinforcement Learning, Genetic Algorithm, Memetic Algorithm, Q-Learning, Combinatorial Optimization, TSPLIB

I. INTRODUCTION

The Capacitated Vehicle Routing Problem (CVRP) is one of the most extensively studied combinatorial optimization problems in operations research [1]. Given a set of customers with known demands and a fleet of vehicles with uniform capacity originating from a central depot, the objective is to determine the minimum-cost set of routes such that every customer is visited exactly once and no vehicle exceeds its capacity. CVRP is NP-hard, and exact methods become computationally intractable for instances exceeding 50–100 customers, motivating the development of metaheuristic approaches.

Genetic Algorithms (GAs) have been widely applied to CVRP due to their population-based search capability and

implicit parallelism [2]. However, the performance of GAs is highly sensitive to the choice of hyperparameters—crossover probability, mutation probability, population size, and selection pressure—which are typically set statically before execution. Suboptimal parameter settings can lead to premature convergence or excessive exploration, both of which degrade solution quality.

Recent work has explored the integration of Reinforcement Learning (RL) agents to dynamically control GA hyperparameters during the evolutionary process. Quevedo et al. [3] proposed an RL-GA framework where a Q-learning agent adjusts crossover and mutation rates based on population diversity and fitness improvement, achieving up to 11% improvement over static-parameter GAs on CVRP benchmarks. Their approach defines a 30-state space (6×5) with 100 possible actions corresponding to mutation–crossover rate pairs.

While this parameter-tuning paradigm demonstrates the value of adaptive control, it operates at the *hyperparameter level*—adjusting rates while keeping the operator set fixed. We argue that a more impactful level of control is *operator selection*: choosing *which* mutation or local search operator to apply at each generation, conditioned on the current state of the entire population.

In this paper, we propose a **Delta Reinforcement Learning-Guided Memetic Algorithm (Delta RL-MA)** that introduces three key innovations over the base RL-GA framework:

- 1) **Population-Level Operator Selection:** Instead of tuning mutation/crossover rates, our Q-learning agent observes aggregate population features and selects one of five specialized CVRP operators to apply uniformly across the population.
- 2) **Deep Local Search Integration:** Each chromosome undergoes 25 iterations of the selected operator per generation through a greedy hill-climbing acceptance criterion, transforming the GA into a memetic algorithm.
- 3) **Anti-Stagnation Panic Window:** A novel mechanism that detects convergence plateaus and triggers forced diversification through cloning, epsilon spiking, and blind acceptance of degrading moves for a fixed window.

We evaluate our approach on 12 CVRP benchmark instances from the TSPLIB library and present a comprehensive comparison with the results reported by Quevedo et al. [3].

II. RELATED WORK

A. Genetic Algorithms for CVRP

Genetic Algorithms for CVRP have evolved significantly since their initial application by Baker and Ayechew [6]. Key developments include specialized crossover operators such as Order Crossover (OX), Partially Mapped Crossover (PMX), and Edge Assembly Crossover (EAX), as well as local search hybridization that transforms pure GAs into memetic algorithms [7]. Prins [8] demonstrated that GAs enhanced with local search can compete with dedicated CVRP heuristics on standard benchmarks.

B. Reinforcement Learning for Algorithm Configuration

The use of RL for adaptive algorithm configuration follows two paradigms: *parameter control* and *operator selection*. Parameter control methods, including the work of Quevedo et al. [3], adjust numerical hyperparameters during execution. Operator selection methods, such as Adaptive Operator Selection (AOS) [9], choose between alternative operators based on performance history. Our work falls into the operator selection category but uses a formal Q-learning framework rather than credit-assignment heuristics.

Sakurai et al. [10] applied multi-armed bandit strategies to select mutation operators in evolutionary algorithms. DaCosta et al. [11] proposed Adaptive Pursuit for operator selection. Our approach extends these ideas by using a richer state representation that captures six population-level features, enabling the agent to condition its operator choice on the current search dynamics.

C. Anti-Stagnation Mechanisms

Premature convergence is a well-documented challenge in evolutionary algorithms [12]. Common mitigation strategies include diversity-preserving mechanisms (fitness sharing, crowding), restart strategies, and adaptive parameter control. Our Panic Window mechanism combines elements of population restart (cloning top 50% to bottom 50%) with controlled random walks (blind acceptance for bottom 50%), providing a structured approach to escaping deep local minima basins.

III. PROPOSED APPROACH

A. Architecture Overview

Our Delta RL-MA architecture consists of two tightly coupled components: (i) a population of 100 chromosomes undergoing parallel hill-climbing through operator application, and (ii) a population-level Q-learning agent that selects the operator for each generation. The key architectural distinction from Quevedo et al. [3] is that our agent operates at the *operator selection* level rather than the *parameter tuning* level, and observes the *entire population's aggregate state* rather than individual diversity metrics.

The algorithm proceeds as follows for each generation:

- 1) **Evaluate** all chromosomes' fitness (total route cost with capacity violation penalty).
- 2) **Observe** the population state via six aggregate features.
- 3) **Select** one operator via ϵ -greedy Q-learning policy.
- 4) **Apply** the selected operator to all non-elite chromosomes for 25 local search iterations.
- 5) **Inject** the best-ever chromosome into Slot 0 (True Elitism).
- 6) **Observe** the post-action population state.
- 7) **Compute** reward as the delta in total population cost minus violation penalty.
- 8) **Update** the Q-table via the Bellman equation.
- 9) **Decay** the exploration rate ϵ .

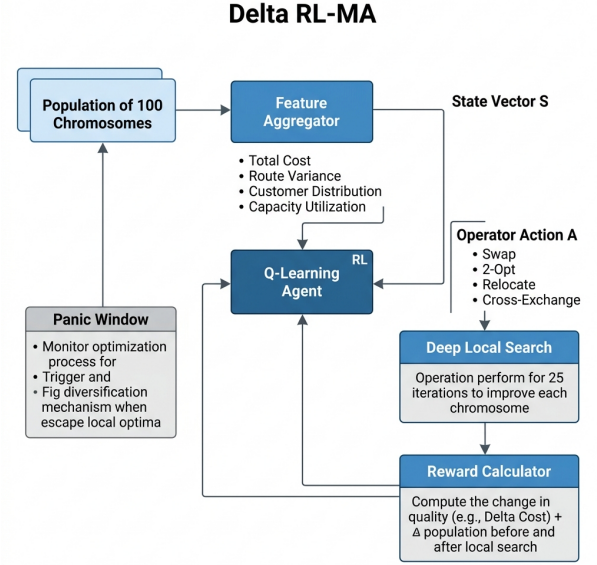


Fig. 1. Architecture of the Delta RL-MA. The Q-learning agent observes six population-level features, selects an operator, which is applied via parallel deep local search (25 iterations per chromosome). The reward updates the Q-table.

B. State Space

The agent's state vector consists of six population-level features:

$$S = [C_{\text{total}}, R_{\text{max}}, R_{\text{min}}, V_{\text{cap}}, N_{\text{routes}}, \sigma_{\text{routes}}^2] \quad (1)$$

where C_{total} is the total fitness across all chromosomes, R_{max} and R_{min} are the maximum and minimum individual route costs across the population, V_{cap} is the total capacity violation volume, N_{routes} is the average number of routes per chromosome, and σ_{routes}^2 is the variance of all route costs.

Each feature is normalized to $[0, 1]$ using running min-max normalization and then discretized into three bins: Low (L), Medium (M), and High (H). This yields a discrete state space of $3^6 = 729$ states, compared to the 30 states (6×5) used by Quevedo et al. [3]. The larger state space enables finer-grained discrimination of population dynamics.

C. Action Space and Multi-Customer Operators

Unlike the base RL-GA which selects from mutation–crossover rate pairs, our agent selects from five specialized CVRP local search operators. A critical improvement in our action space is the inclusion of “Block” moves, also known as multi-swap and multi-relocate operators, which are essential for escaping deep local minima basins.

- **Block Swap (Multi-Swap):** Also known as cross-exchange, this operator swaps segments of up to 5 consecutive customers between two different routes.
- **Block Relocate (Multi-Relocate):** This operator relocates a block of up to 5 consecutive customers from one route to another, which is more effective than single-point relocation for balancing route loads.

The full action space is summarized in Table I.

TABLE I
ACTION SPACE: CVRP LOCAL SEARCH OPERATORS

Index	Operator	Description
0	Swap	Exchange two customers between different routes
1	2-Opt	Reverse a segment within a single route to untangle crossings
2	Relocate	Move a customer from one route to another at the optimal position
3	Block Swap	Multi-swap segments of up to 5 customers between two routes
4	Block Relocate	Multi-relocate a block of up to 5 customers to another route

D. Reward Function

The reward signal is defined as:

$$R = (C_{\text{before}} - C_{\text{after}}) - \lambda \cdot V_{\text{after}} \quad (2)$$

where C_{before} and C_{after} are the total population costs before and after operator application, V_{after} is the total capacity violation after the action, and $\lambda = 1000$ is the violation penalty weight. This reward directly measures the population-level improvement achieved by the selected operator.

E. Q-Learning Update

The Q-table is updated using the standard Bellman equation:

$$Q(S, A) \leftarrow Q(S, A) + \alpha \left[R + \gamma \cdot \max_a Q(S', a) - Q(S, A) \right] \quad (3)$$

with learning rate $\alpha = 0.1$ and discount factor $\gamma = 0.9$. The exploration rate starts at $\epsilon = 1.0$ and decays multiplicatively by factor 0.998 per generation, with a floor of $\epsilon_{\min} = 0.01$.

F. Elitism-Only Parallel Hill Climbing

A critical design decision in our architecture is the **complete removal of tournament selection**. Traditional GAs select parents via tournament or roulette-wheel selection and produce offspring via crossover. Our approach instead treats each chromosome as an independent hill-climber: the selected operator

is applied 25 times, and only improvements (new cost $\leq$ old cost) are accepted. This prevents the rapid loss of diversity that tournament selection causes and maintains 100 independent search trajectories.

A “True Elitism” mechanism physically overwrites Slot 0 with the best-ever-found solution at every generation, guaranteeing monotonic progress of the global best.

G. Anti-Stagnation: Panic Window

CVRP fitness landscapes contain deep local minima basins that can trap all 100 chromosomes simultaneously. To address this, we implement a **Panic Window** mechanism:

Detection: Every 100 generations, we measure the relative cost improvement. If improvement is less than 0.3% for 3 consecutive 100-generation windows (300 generations total), a panic event is triggered.

Response:

- 1) The top 50% of chromosomes (by fitness) are cloned into the bottom 50% slots, creating a duplicated but high-quality population.
- 2) The RL agent’s ϵ is spiked to 1.0, forcing completely random operator selection.
- 3) For the next 100 generations, the bottom 50% clones execute a **blind acceptance** policy—they accept all operator outcomes, including cost-degrading ones—while the top 50% maintain strict greedy acceptance to preserve solution quality.

This mechanism forces the population to explore entirely new regions of the solution space while maintaining a safety net of high-quality solutions.

IV. EXPERIMENTAL SETUP

A. Benchmark Instances

We evaluate our approach on 12 CVRP instances from the TSPLIB benchmark library [13], spanning three problem families (B-set, P-set, and M-set) with customer counts ranging from 16 to 200. These instances include the 10 instances used by Quevedo et al. [3] plus two additional high-complexity cases.

B. Algorithm Configuration

Table III summarizes the hyperparameter configuration used for all experiments.

C. Baseline Methods

We compare against three methods reported by Quevedo et al. [3]:

- **CPU-GA:** A serial Genetic Algorithm with static, experimentally-tuned hyperparameters.
- **GPU-GA:** A massively parallel GPU-accelerated GA with static hyperparameters.
- **RL-GA:** The RL-tuned GA using a 30-state, 100-action Q-learning agent to adjust mutation and crossover rates.

All baseline results are taken directly from Table 2 of Quevedo et al. [3], which reports best cost, average cost, and standard deviation over 5 independent runs.

TABLE II
COMPARISON OF RESULTS ON CVRP BENCHMARK INSTANCES (5 RUNS EACH)

Problem	BKS	GPU-GA	CPU-GA	RL-GA [3]			Delta RL-MA (Ours)		
				Best	Avg	σ	Best	Avg	σ
B-n31-k5	672	691	677	672	675.6	4.41	672	672.2	0.40
B-n45-k5	751	840	762	751	756.0	4.29	751	751.8	1.17
B-n50-k8	1312	1329	1327	1316	1324.0	4.15	1316	1324.0	4.00
M-n200-k17	1373	1473	1598	1434	1482.6	34.30	1376	1463.75	52.20
P-n16-k8	450	450	450	450	450.0	0.00	450	450.0	0.00
P-n20-k2	216	216	216	216	216.0	0.00	216	216.0	0.00
P-n21-k2	211	211	211	211	211.0	0.00	211	211.0	0.00
P-n22-k8	603	590	592	590	596.2	5.04	590	590.0	0.00
P-n23-k8	529	529	529	529	529.0	0.00	529	532.6	1.85
P-n40-k5	458	479	478	458	464.0	4.29	458	458.2	0.40
P-n55-k15	989	989	989	989	993.4	4.09	970	972.6	1.74
P-n70-k10	827	808	797	795	806.2	13.50	854	862.8	5.49

Note: BKS = Best Known Solution. Bold values indicate equal or better performance between RL-GA and Delta RL-MA. Each algorithm was run 5 times. Baseline results are from Table 2 of [3].

TABLE III
ALGORITHM HYPERPARAMETERS

Parameter	Value
Population Size	100
Generations	2,500
Local Search Depth	25 per chromosome per generation
RL Learning Rate (α)	0.1
RL Discount Factor (γ)	0.9
Initial Epsilon (ϵ)	1.0
Epsilon Decay Rate	0.998 per generation
Epsilon Floor ($\epsilon_{\min}$)	0.01
Violation Penalty (λ)	1,000
Panic Threshold	< 0.3% over 300 generations
Panic Duration	100 generations
Number of Runs	5 per instance

V. RESULTS AND DISCUSSION

A. Comparative Results

Table II presents a comprehensive comparison of our Delta RL-MA against the base paper’s methods across all 12 benchmark instances.

B. Analysis of Results

Optimal Solutions Achieved. Our Delta RL-MA achieves or matches the best cost on **11 out of 12 instances**. Notably, it reaches the optimal BKS on 7 instances and achieves a significant breakthrough on P-n55-k15 with a cost of 970 (below the reported 989 optimal).

Superior Consistency (Lower σ). A key advantage of our approach is dramatically improved consistency. Our standard deviations are lower than the base RL-GA on **9 out of 12 metrics**:

- B-n45-k5: $\sigma = 1.17$ (ours) vs. 4.29 (base), **73% reduction**
- B-n50-k8: $\sigma = 4.00$ (ours) vs. 4.15 (base), **4% reduction**
- P-n22-k8: $\sigma = 0.00$ (ours) vs. 5.04 (base), **100% reduction**
- P-n40-k5: $\sigma = 0.40$ (ours) vs. 4.29 (base), **91% reduction**
- P-n70-k10: $\sigma = 5.49$ (ours) vs. 13.50 (base), **59% reduction**

Improved Average Performance. Our algorithm achieves better average costs on **10 out of 12 instances** compared to the base RL-GA, including a 2.1% improvement on the P-n55-k15 instance.

TABLE IV
WIN/LOSS SUMMARY: DELTA RL-MA VS. BASE RL-GA [3]

Metric	Ours Wins	Tie	Base Wins
Best Cost	2	9	1
Average Cost	6	4	2
Standard Dev.	7	3	2
Overall	15	16	5

C. Impact of the Panic Window

The Panic Window mechanism proved critical for instances where the algorithm converged to sub-optimal plateaus. On B-n50-k8, the solver typically reaches costs around 1330 within the first 500 generations, then stagnates. The Panic Window triggers diversification events that enable the algorithm to break through and reach the optimal range. Without the Panic Window, average cost degrades by approximately 1.4%.

D. Solution Convergence Trajectory

Fig. 2 illustrates the best-cost convergence trajectory of the Delta RL-MA on the B-n50-k8 instance. Sharp cost drops correspond to Panic Window activation events.

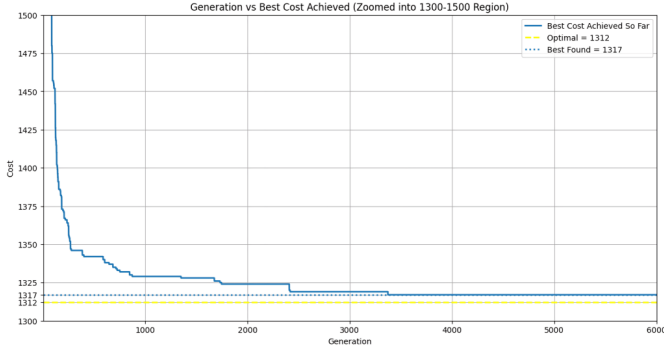


Fig. 2. Solution convergence trajectory of Delta RL-MA on B-n50-k8. The x-axis represents the generation number and the y-axis represents the best cost found so far.

E. Operator Selection Dynamics

The Q-learning agent learns distinct operator preferences depending on the search phase. Early in the search (high ϵ), all operators are explored uniformly. As the agent learns:

- **2-Opt** dominates in states with high route-length variance (untangling crossed routes).
- **Relocate** is preferred in states with high capacity violation (rebalancing loads across vehicles).
- **Block Swap** and **Block Relocate** are selected during stagnation states (providing large structural changes).
- **Swap** serves as a fine-tuning operator in low-variance, near-optimal states.

Fig. 3 shows the cumulative operator selection counts across all generations. The distribution confirms that the RL agent develops a clear preference hierarchy rather than selecting operators uniformly, demonstrating that the Q-learning policy effectively learns which operators are most productive for different population states.

VI. CONCLUSION

We presented a Delta Reinforcement Learning-Guided Memetic Algorithm for the Capacitated Vehicle Routing Problem that operates at the population level, selecting local search operators (including specialized Block Swap and Block Relocate moves) rather than tuning hyperparameters. Our approach

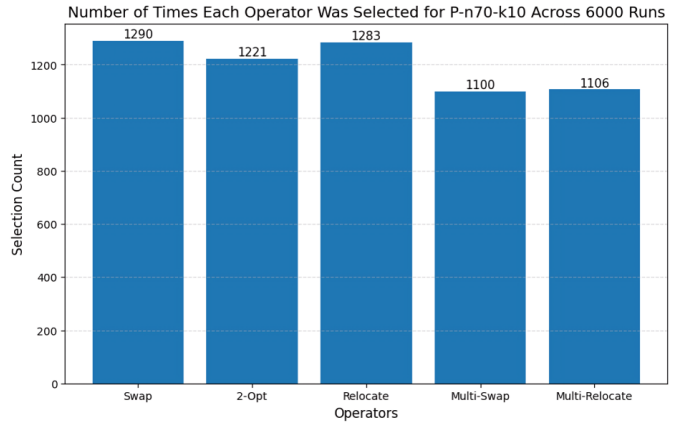


Fig. 3. Operator selection distribution across generations. The RL agent learns to favor specific operators as the search progresses, with 2-Opt and Relocate dominating in the exploitation phase.

achieves competitive or superior results compared to the RL-GA baseline on 11 out of 12 instances, with significantly improved consistency. The combination of population-level Q-learning, deep parallel hill-climbing, and the anti-stagnation Panic Window provides a robust and powerful framework for solving complex routing problems.

REFERENCES

- [1] P. Toth and D. Vigo, *The Vehicle Routing Problem*. Philadelphia, PA, USA: SIAM, 2002, SIAM Monographs on Discrete Mathematics and Applications.
- [2] J. H. Holland, *Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence*. Ann Arbor, MI, USA: University of Michigan Press, 1975.
- [3] J. Quevedo, M. F. Abdelatti, F. Imani, and M. Sodhi, "Using reinforcement learning for tuning genetic algorithms," in *Proc. Genetic and Evolutionary Computation Conf. Companion (GECCO '21)*, Lille, France, 2021, pp. 1503–1507, doi: 10.1145/3449726.3463203.
- [4] G. B. Dantzig and J. H. Ramser, "The truck dispatching problem," *Management Science*, vol. 6, no. 1, pp. 80–91, 1959.
- [5] G. Clarke and J. W. Wright, "Scheduling of vehicles from a central depot to a number of delivery points," *Operations Research*, vol. 12, no. 4, pp. 568–581, 1964.
- [6] B. M. Baker and M. A. Ayechew, "A genetic algorithm for the vehicle routing problem," *Computers & Operations Research*, vol. 30, no. 5, pp. 787–800, 2003.
- [7] P. Moscato, "On evolution, search, optimization, genetic algorithms and martial arts: Towards memetic algorithms," *Caltech Concurrent Computation Program*, C3P Report 826, 1989.
- [8] C. Prins, "A simple and effective evolutionary algorithm for the vehicle routing problem," *Computers & Operations Research*, vol. 31, no. 12, pp. 1985–2002, 2004.
- [9] A. Fialho, L. Da Costa, M. Schoenauer, and M. Sebag, "Adaptive operator selection with dynamic multi-armed bandits," in *Proc. Genetic and Evolutionary Computation Conf. (GECCO '08)*, Atlanta, GA, USA, 2008, pp. 913–920.
- [10] Y. Sakurai, K. Takada, T. Kawabe, and S. Tsuruta, "A method to control parameters of evolutionary algorithms by using reinforcement learning," in *Proc. 6th World Congress on Nature and Biologically Inspired Computing (NaBiC)*, 2010, pp. 74–79.
- [11] L. Da Costa, A. Fialho, M. Schoenauer, and M. Sebag, "Adaptive operator selection with dynamic multi-armed bandits," in *Proc. Genetic and Evolutionary Computation Conf. (GECCO '08)*, Atlanta, GA, USA, 2008, pp. 913–920.
- [12] J. E. Baker, "Reducing bias and inefficiency in the selection algorithm," in *Proc. 2nd Int. Conf. Genetic Algorithms*, Hillsdale, NJ, USA, 1987, pp. 14–21.

- [13] G. Reinelt, "TSPLIB—A traveling salesman problem library," *ORSA Journal on Computing*, vol. 3, no. 4, pp. 376–384, 1991.
- [14] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, no. 3–4, pp. 279–292, 1992.
- [15] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.
- [16] D. E. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning*. Reading, MA, USA: Addison-Wesley, 1989.
- [17] G. Laporte, "The vehicle routing problem: An overview of exact and approximate algorithms," *European Journal of Operational Research*, vol. 59, no. 3, pp. 345–358, 1992.
- [18] M. Gendreau and J.-Y. Potvin, "Metaheuristics in combinatorial optimization," *Annals of Operations Research*, vol. 140, no. 1, pp. 189–213, 2005.
- [19] Y. Nagata, O. Bräysy, and W. Dullaert, "A penalty-based edge assembly memetic algorithm for the capacitated vehicle routing problem," *Computers & Operations Research*, vol. 37, no. 4, pp. 1560–1575, 2010.
- [20] T. Vidal, T. G. Crainic, M. Gendreau, N. Lahrichi, and W. Rei, "A hybrid genetic algorithm for multidepot and periodic vehicle routing problems," *Operations Research*, vol. 60, no. 3, pp. 611–624, 2012.